Konfiguracja dostępów do maszyny wirtualnej

Cel zadania:

Celem zadania jest skonfigurowanie różnych metod dostępu do lokalnej maszyny wirtualnej z Debianem, w tym SSH z uwierzytelnianiem kluczami asymetrycznymi, kontrolą uprawnień użytkowników i grup, ograniczonym sudo oraz dodatkowymi zabezpieczeniami. Dodatkowo należy przetestować alternatywne metody dostępu, takie jak konsola szeregowa, VSOCK i panel administracyjny Cockpit.

1. Podstawowy SSH
-Utworzyć użytkownika i nadać mu uprawnienia sudo
-Skonfigurować logowanie poprzez kryptografie asymetryczna (SSH)
-Prezentacja działania skonfigurowanej autentykacji logując się na utworzonego użytkownika

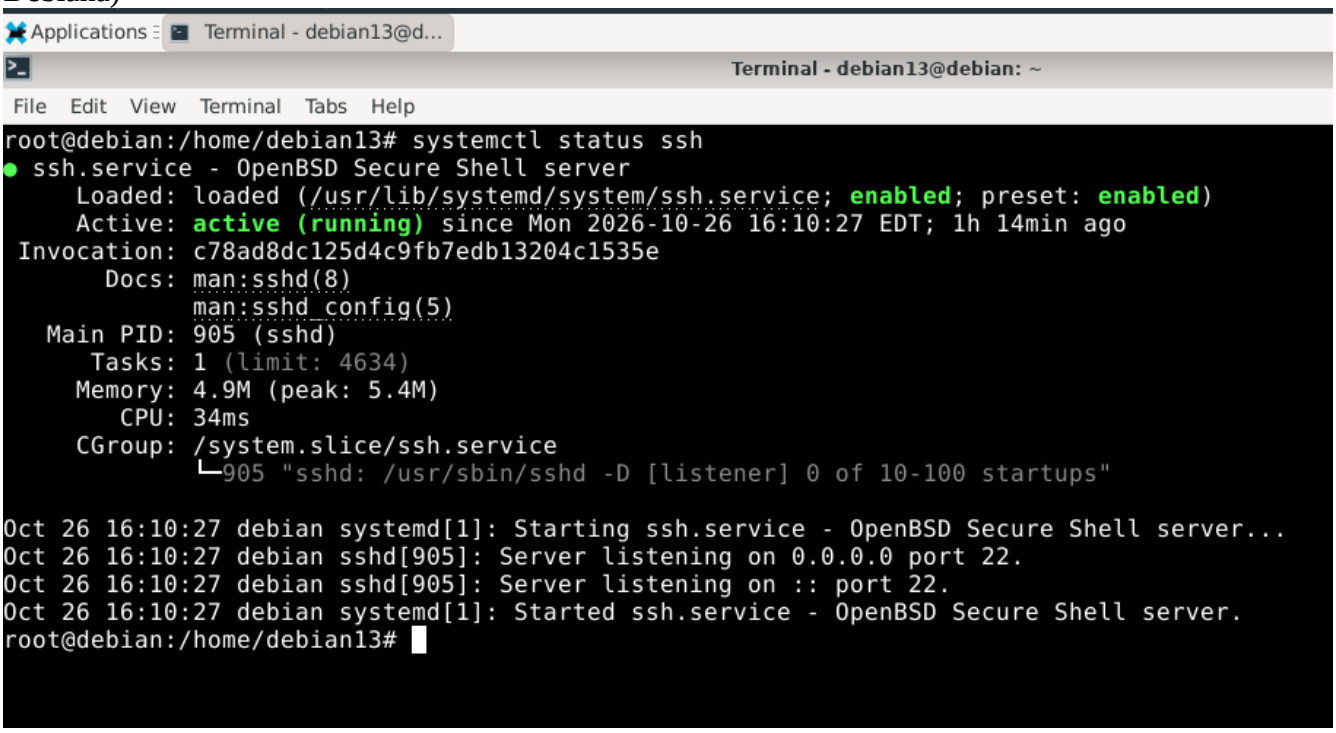
2. Zaawansowana konfiguracja SSH
-Stworzymy osobny plik konfiguracyjny aby uniknąć konfilktu z domyślną konfiguracją podczas aktualizacji serwera SSH
-Skonfigurujemy serwer SSH tak aby różne “funkcjonalności” wymagały przypisania odpowiednich grup użytkownikowi.
-Sprawdzamy jakie algorytmy kryptograficzne oferuje serwer oraz wyłączamy te przestarzałe
-Ustawimy niestandardowy port SSH
-Skonfigurujemy blokowanie problematycznych adresów IP (te które nie potrafią się poprawnie uwierzytelnić) i dodamy odpowiedni zakres adresów IP do listy “zaufanych”

3. Ograniczenia Sudo
-Tworzymy kolejnego użytkownika i skonfigurujemy sudo tak aby mógł wykonywać tylko jedną komendę z uprawnieniami sudo
-Zezwolimy temu użytkownikowi na wykonywanie innej komendy, jako inny użytkownik, bez podawania hasła
-Pokażemy działanie dokonanych zmian

4. Alternatywne sposoby dostępu
-Dodamy do maszyny wirtualnej port seryjny, ustawimy system tak aby wyświetlał na tym porcie konsole oraz zalogujemy się przez tą konsolę
-Dołączymy maszynę do sieci VSOCK
-Skonfigurujemy cockpit (webowy panel administracyjny) i zalogujemy się do niego oraz pokażemy jego możliwości

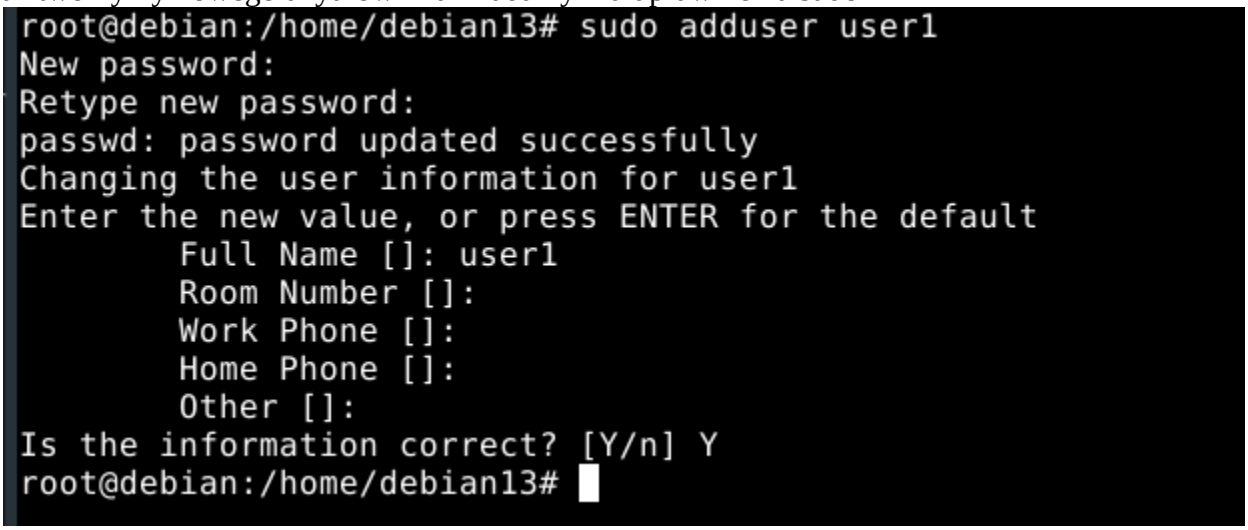
1. Podstawowy SSH

Sprawdzamy najpierw czy nasz serwer SSH działa (zainstalowałem go odrazu podczas instalacji Debiana)

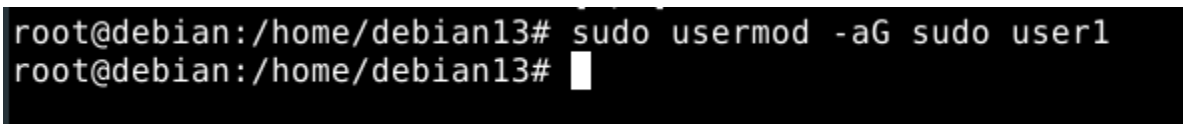


Jak widać jest aktywny

Teraz tworzymy nowego użytkownika i nadamy mu uprawnienia sudo



Dodajemy nowego użytkownika “user1”



Nadajemy mu uprawnienia sudo

Sprawdzmy teraz dla pewności czy użytkownik faktycznie ma uprawnienia sudo

```
root@debian:/home/debian13# id user1
uid=1001(user1) gid=1001(user1) groups=1001(user1),27(sudo),100(users)
root@debian:/home/debian13# groups user1
user1 : user1 sudo users
root@debian:/home/debian13#
```

Jak widać tak (sudo users)

Teraz skonfigurujemy autentykację nowego użytkownika za pomocą kryptografii asymetrycznej

```
root@debian2:/home/debian2# ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:xOBx4EgjlGPLkghfqmwZ8rbvBYzPaEdFQDWyf6DrFy4 root@debian2
The key's randomart image is:
+--[ED25519 256]--+
| .oo*oB..        |
|. +o.O *         |
|o= =o = o        |
|=.=o + o         |
|o++ = . S        |
|.+o= o..         |
|..o.=...         |
| ..oE.o          |
|   o+o           |
+----[SHA256]-----+
```

Generujemy klucz na maszynie wirtualnej z której będziemy się łączyć

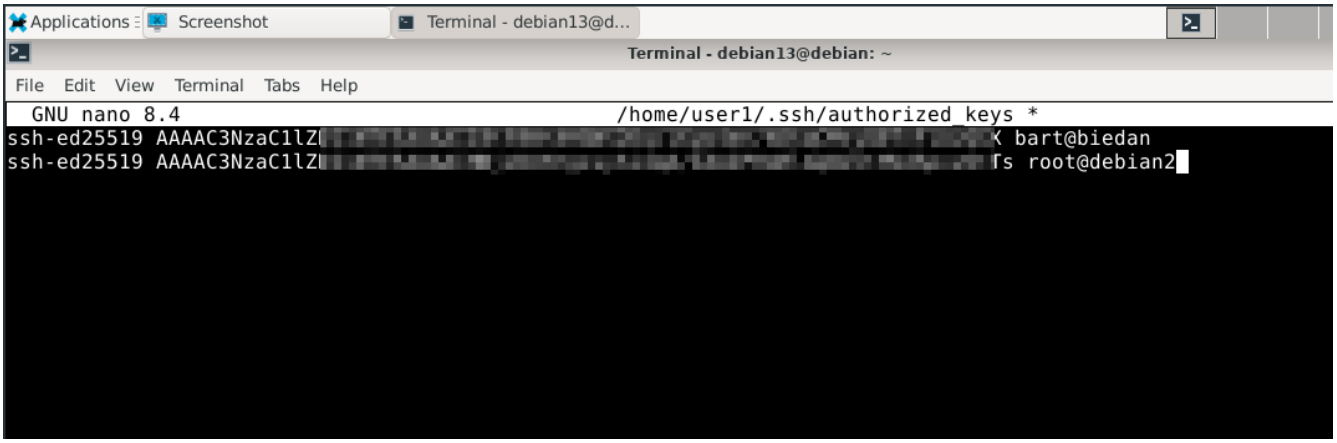
```
root@debian:/home/debian13# sudo mkdir -p /home/user1/.ssh
root@debian:/home/debian13# chmod 700 /home/user1/.ssh
root@debian:/home/debian13#
```

```
Your identification has been saved in publicvm_key
Your public key has been saved in publicvm_key.pub
The key fingerprint is:
SHA256:qHv7p5NM7sMZYrfb8IcplIuZUXM7JUWoHdzP8RP3iYU bart@biedan
The key's randomart image is:
+--[ED25519 256]--+
|          . +. .  |
|           + oE.o.|
|          o o ooo=|
|        .+ + ..ooo|
|       ..S+ +    .|
|      .+ * o      |
|     .. &.* +     |
|      .= %++ .    |
|     ...+=Bo.     |
+----[SHA256]-----+
```

Wygenerowałem również klucz na moim laptopie do połączenia

Klucz który służy do połączenia to klucz publiczny wymieniamy się nim z hostem który oferuje usługę.

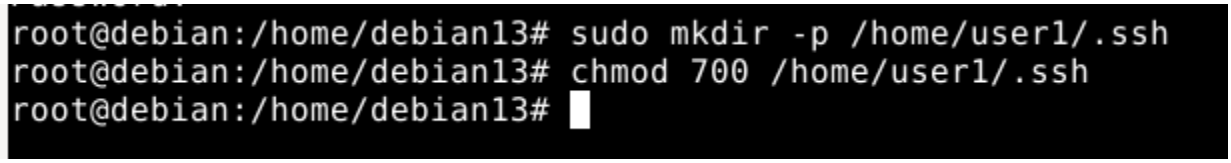
Teraz tworzymy plik w /user1/.ssh/authorized_keys i dodajemy klucze publiczne z laptopa oraz drugiej maszyny wirtualnej



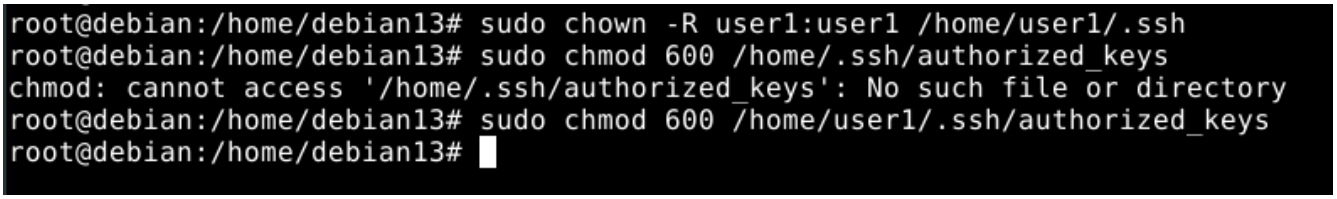
Mamy błąd, nie mamy folderu użytkownika na klucze i konfiguracje ssh



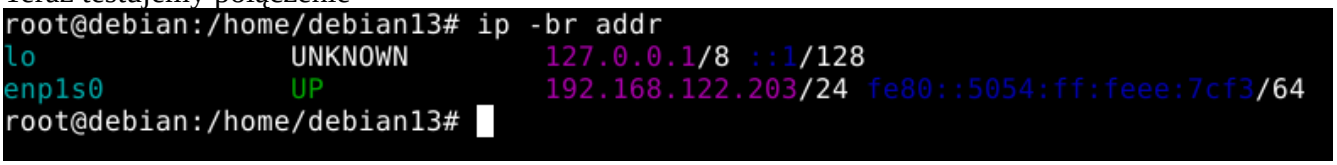
Zatem tworzymy katalog i nadajemy mu odpowiednie uprawnienia



Następnie ustawiamy właściciela (na user1) i uprawnienia do plików SSH (aby serwer mógł bezpiecznie odczytywać klucze oraz inni użytkownicy nie mogli ich modyfikować)

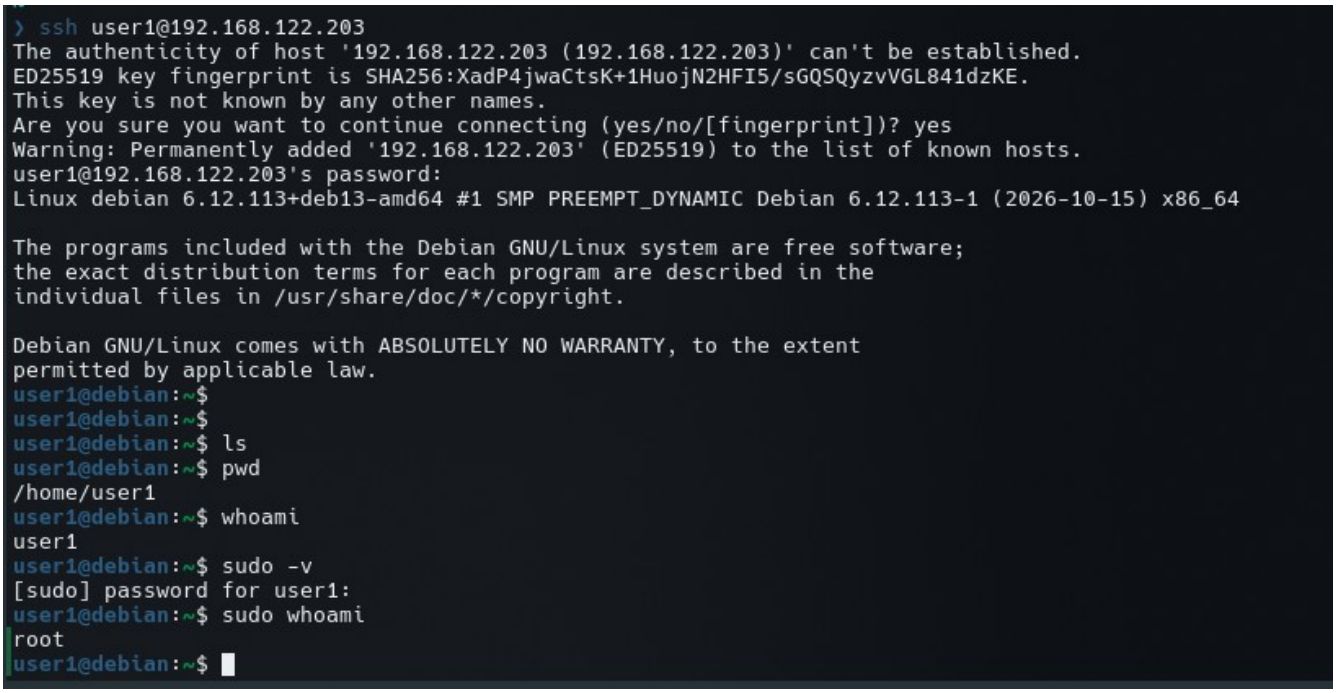


Teraz testujemy połączenie

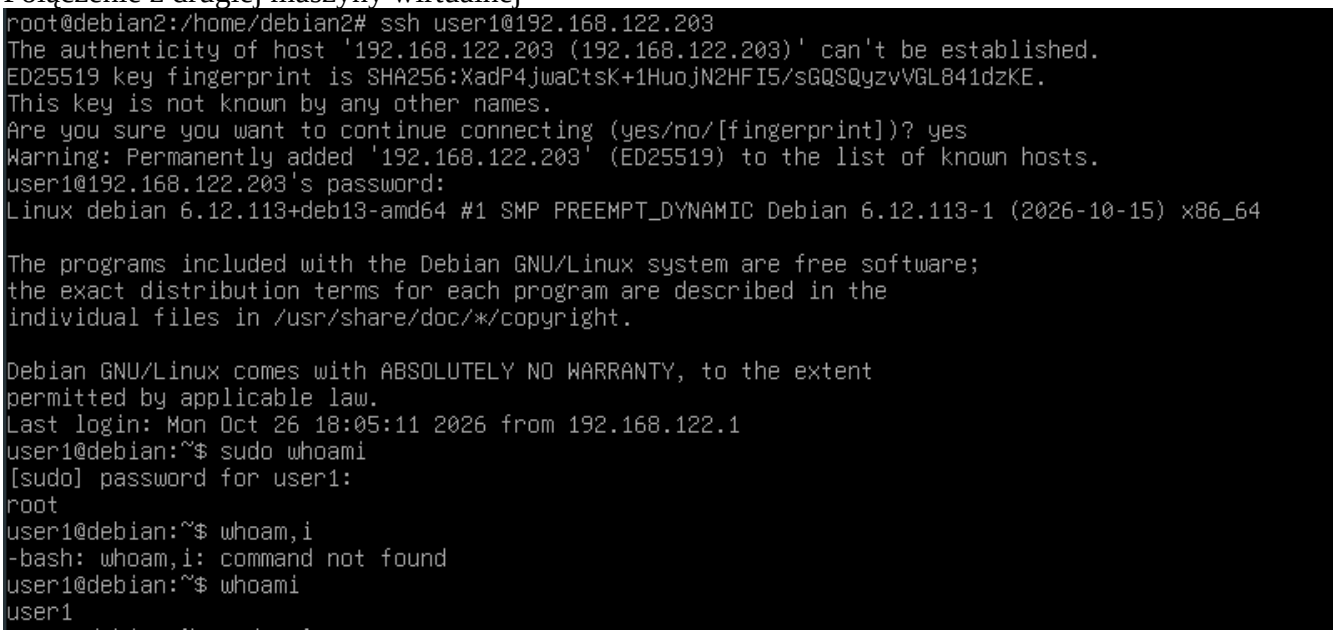


Sprawdzamy adres IP

Połączenie z laptopa



Połączenie z drugiej maszyny wirtualnej



Jak widać wszystko działa tak jak powinno

2. Zaawansowana konfiguracja SSH

Sprawdzamy czy SSH wczytuje dodatkowe pliki, ponieważ stworzymy nowy plik konfiguracyjny który nie będzie kolidować z domyślną konfiguracją.

```
root@debian:/home/debian13# grep -n "Include" /etc/ssh/sshd_config
12:Include /etc/ssh/sshd_config.d/*.conf
root@debian:/home/debian13#
```

Jak widać zawiera “Include”, więc powinien wczytać naszą nową konfigurację

Teraz stworzymy grupy do odpowiednich “funkcjonalności”, aby móc potem dodawać użytkownika do tych grup (ułatwi to zarządzanie dostępem użytkownika do możliwości korzystania z różnych funkcjonalności).

```
root@debian:/home/debian13# sudo groupadd ssh-login
root@debian:/home/debian13# sudo groupadd ssh-password-selected-ip
root@debian:/home/debian13# sudo groupadd ssh-password-any-ip
root@debian:/home/debian13# sudo groupadd ssh-port-forward
root@debian:/home/debian13# sudo groupadd ssh-gateway-forward
root@debian:/home/debian13# sudo groupadd ssh-x11
root@debian:/home/debian13#
```

ssh-login = logowanie do ssh

ssh-password-selected-ip = logowanie się hasłem z wybranych adresów IP

ssh-password-any-ip = logowanie się hasłem z dowolnego IP

ssh-port-forward = umożliwienie przekazywania portów

ssh-gateway-forward = pozwala przekazać port na serwer, ustawiając go w tryb nasłuchu na wszystkich adresach

ssh-x11 = pozwala korzystać z przekazywania X11

Dodamy sobie odrazu dwóch innych użytkowników dla testów

```
root@debian:/home/debian13# sudo adduser user2
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user2
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] Y
root@debian:/home/debian13# sudo adduser user3
New password:
Retype new password:
No password has been supplied.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user3
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
```

Przypisujemy użytkowników do grup.
User1 = Logowanie ssh, przekazywanie portów
User2 = Logowanie ssh, logowanie hasłem z wybranych adresów IP
User3 = Logowanie ssh, logowanie hasłem z dowolnego adresu IP, przekazywanie portów i przekazywanie portów z nasłuchem na wszystkich adresach i X11

```
root@debian:/home/debian13# sudo usermod -aG ssh-login user1
root@debian:/home/debian13# sudo usermod -aG ssh-port-forward user1
root@debian:/home/debian13#
root@debian:/home/debian13# sudo usermod -aG ssh-login user2
root@debian:/home/debian13# sudo usermod -aG ssh-password-selected-ip user2
root@debian:/home/debian13#
root@debian:/home/debian13# sudo usermod -aG ssh-login user3
root@debian:/home/debian13# sudo usermod -aG ssh-password-any-ip user3
root@debian:/home/debian13# sudo usermod -aG ssh-port-forward user3
root@debian:/home/debian13# sudo usermod -aG ssh-gateway-ports user3
usermod: group 'ssh-gateway-ports' does not exist
root@debian:/home/debian13# sudo usermod -aG ssh-gateway-forward user3
root@debian:/home/debian13# sudo usermod -aG ssh-x11 user3
root@debian:/home/debian13#
root@debian:/home/debian13#
```

Sprawdzamy przypisania

```
root@debian:/home/debian13# groups user1
user1 : user1 sudo users ssh-login ssh-port-forward
root@debian:/home/debian13# groups user2
user2 : user2 users ssh-login ssh-password-selected-ip
root@debian:/home/debian13# groups user3
user3 : user3 users ssh-login ssh-password-any-ip ssh-port-forward ssh-gateway-forward ssh-x11
root@debian:/home/debian13#
```

Wszystko się zgadza.

Przejdziemy teraz do stworzenia nowej konfiguracji, która będzie zawierać przypisania odpowiednich grup użytkownika do danych “funkcjonalności”.

```
  GNU nano 8.4                    /etc/ssh/sshd_config.d/1-myconfig.conf *
#Default config
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
AllowGroups ssh-login
AllowTcpForwarding no
GatewayPorts no
X11Forwarding no
```

Domyślna konfiguracja powinna wyglądać w ten sposób

PermitRootLogin – zabrania logowania się na konto roota
PubkeyAuthentication – zezwala na logowanie się za pomocą kluczem SSH
PasswordAuthentication – wyłącza logowanie hasłem przez SSH (wyłączamy ze względu że logowanie ma być możliwe tylko dla wybranych grup)
AllowGroups – pozwala logować się tylko użytkownikom należacym do grupy “ssh-login”
AllowTcpForwarding – blokujemy przekazywanie portów
GatewayPorts – blokujemy przekazywanie portów z nasłuchem
X11Forwarding – blokujemy przekazywanie X11

Dodajemy dla “user2” logowanie hasłem z wybranego adresu IP

```
# user2-password only from selected IP
Match Group ssh-password-selected-ip Address 192.168.122.149
        PasswordAuthentication yes
```

Dodajemy dla “user3” logowanie hasłem z dowolnego adresu IP

```
# user3-password from any address IP
Match Group ssh-password-any-ip
        PasswordAuthentication yes
```

Dodajemy przekazywanie portów dla “user1” i “user3”

```
# user1&user3-port forwarding
Match Group ssh-port-forward
        AllowTcpForwarding yes
```

Dodajemy przekazywanie portów z nasłuchem na wszystkich adresach dla “user3”

```
# user3-remote port can listen on all addresses
Match Group ssh-gateway-forward
        GatewayPorts clientspecified
```

Dodajemy przekazywanie X11 dla “user3”

```
# user3-forward X11
Match Group ssh-x11
        X11Forwarding yes
```

Sprawdzamy konfigurację, czy nie zawiera błędów składniowych

```
root@debian:/home/debian13# sudo sshd -t
root@debian:/home/debian13#
```

Nie zwróciło żadnego wyniku, więc konfiguracja nie ma żadnych błędów składniowych

Teraz przeładujemy SSH aby zastosować konfigurację

```
root@debian:/home/debian13# sudo systemctl reload ssh
root@debian:/home/debian13# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-27 15:34:06 EDT; 1h 27min ago
 Invocation: f48aafe03b1f437e9a3adc6d64f3be81
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 834 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 2108 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 2110 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 854 (sshd)
      Tasks: 1 (limit: 4634)
     Memory: 4.9M (peak: 7.4M)
        CPU: 72ms
     CGroup: /system.slice/ssh.service
             └─854 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Testujemy funkcjonalności

Jeszcze dodajemy możliwość logowania się na “user1” hasłem z danego adresu IP (bo w przeciwnym razie nie będziemy w stanie się połączyć, ze względu na to że będzie to wymagało logowania kluczem zamiast hasła, wynika to z ustawienia globalnie logowania bez hasła).

```
+ ❯ ssh user1@192.168.122.203
user1@192.168.122.203: Permission denied (publickey).
```

```
# user1-password only from selected IP
Match User user1 Address 192.168.122.1
        PasswordAuthentication yes
```

Przechodzimy do właściwego testu
Logowanie z podanego IP na “user1”

```
+2 ❯ ssh user1@192.168.122.203
user1@192.168.122.203's password:
Permission denied, please try again.
user1@192.168.122.203's password:
Permission denied, please try again.
user1@192.168.122.203's password:
Linux debian 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (202

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Oct 27 17:59:03 2026 from 192.168.122.1
user1@debian:~$
```

Logowanie z innego IP na “user1”

```
root@debian2:/home/debian2# ssh user1@192.168.122.203
user1@192.168.122.203: Permission denied (publickey).
root@debian2:/home/debian2#
```

Teraz przetestujemy logowanie na “user2” który też wymaga danego IP

```
root@debian2:/home/debian2# ssh user2@192.168.122.203
user2@192.168.122.203's password:
Linux debian 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Oct 27 17:53:07 2026 from 192.168.122.149
user2@debian:~$
```

Logowanie z innego IP na “user2”

```
~ took 10m42s
+2 ❯ ssh user2@192.168.122.203
user2@192.168.122.203: Permission denied (publickey).

~
+2 ❯
```

Logowanie na “user3” który akceptuje dowolne IP

```
+2 ❯ ssh user3@192.168.122.203
user3@192.168.122.203's password:
Linux debian 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Oct 27 17:53:43 2026 from 192.168.122.1
user3@debian:~$
```

```
root@debian2:/home/debian2# ssh user3@192.168.122.203
user3@192.168.122.203's password:
Linux debian 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Oct 27 18:13:28 2026 from 192.168.122.1
user3@debian:~$ _
```

Wszystkie testy logowania zaliczone

Testujemy przekazywanie portów

Włączamy tymczasowy serwer HTTP na porcie 8080

```
root@debian:/home/debian13# python3 -m http.server 8080 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/) ...
```

Uruchamiamy tunel SSH na drugim urządzeniu

```
~
+2 ❯ ssh -i ./publicvm_key -N -L 18080:127.0.0.1:8080 user1@192.168.122.203
```

W kolejnym terminalu na tym samym urządzeniu sprawdzamy odpowiedz serwera

```
❯ curl http://127.0.0.1:18080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_history">.bash_history</a></li>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".config/">.config/</a></li>
<li><a href=".dmrc">.dmrc</a></li>
<li><a href=".face">.face</a></li>
<li><a href=".face.icon">.face.icon@</a></li>
<li><a href=".gnupg/">.gnupg/</a></li>
<li><a href=".ICEauthority">.ICEauthority</a></li>
<li><a href=".local/">.local/</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".Xauthority">.Xauthority</a></li>
<li><a href=".xsession-errors">.xsession-errors</a></li>
<li><a href=".xsession-errors.old">.xsession-errors.old</a></li>
<li><a href="Desktop/">Desktop/</a></li>
<li><a href="Documents/">Documents/</a></li>
<li><a href="Downloads/">Downloads/</a></li>
<li><a href="Music/">Music/</a></li>
<li><a href="Pictures/">Pictures/</a></li>
<li><a href="Public/">Public/</a></li>
<li><a href="Templates/">Templates/</a></li>
<li><a href="Videos/">Videos/</a></li>
</ul>
<hr>
</body>
</html>

~
❯
```

Mamy odpowiedz, więc nasze przekierowanie portu działa

Spróbujmy potwórzyć to na użytkowniku “user2”

```
~ took 3m39s
+3 ❯ ssh -i ./publicvm_key -N -L 18080:127.0.0.1:8080 user2@192.168.122.203
user2@192.168.122.203: Permission denied (publickey).

~
+3 ❯
```

Jak widać nie możemy uruchomić tunelu

Sprawdzmy przekazywanie portów z nasłuchem na wszystkich adresach

Uruchamiamy usługę testową na urządzeniu

```
+3 ❯ python3 -m http.server 8081 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8081 (http://127.0.0.1:8081/) ...
```

Tworzymy zdalny tunel jako “user3”, na tym samym urządzeniu w drugim terminalu

```
❯ ssh -N -R 0.0.0.0:18081:127.0.0.1:8081 user3@192.168.122.203
user3@192.168.122.203's password:
```

Sprawdzamy nasłuch na naszym serwerze

```
root@debian:/home/debian13# sudo ss -lntp | grep 18081
LISTEN 0      128          0.0.0.0:18081      0.0.0.0:*    users:(("sshd-session",pid=3770,fd=8))
root@debian:/home/debian13#
```

Nasłuch działa.

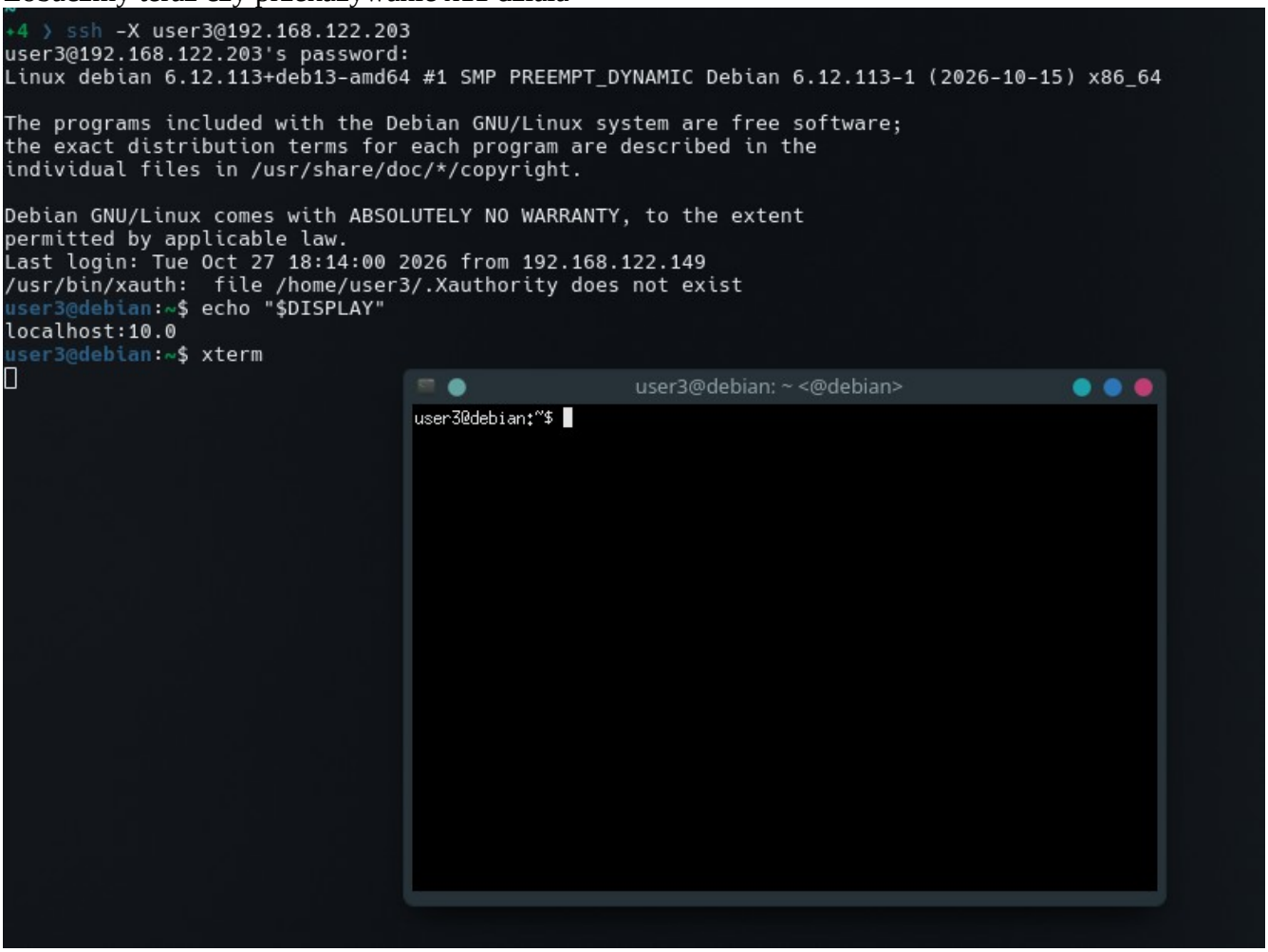
Spróbujmy powtórzyć tą operacje dla “user2”

```
~ took 2m31s
+ ❯ ssh -N -R 0.0.0.0:18081:127.0.0.1:8082 user2@192.168.122.203
user2@192.168.122.203: Permission denied (publickey).

~
+ ❯
```

Jak widać nie mamy takiej możliwości, zgodnie z założeniem.

Zobaczmy teraz czy przekazywanie x11 działa



Jak widać przekazywanie x11 działa, mamy przekierowanie wyświetlacza “localhost:10.0” oraz otworzyliśmy xterma.

Spróbujmy powtórzyć ten proces dla użytkownika “user2”



Oczywiście jest to niemożliwe.

Sprawdzimy jakie algorytmy kryptograficzne oferuje serwer oraz wyłączymy te przestarzałe

```
root@debian:/home/debian13# ssh -V
OpenSSH_10.0p2 Debian-7+deb13u4, OpenSSL 3.5.7 9 Jun 2026
root@debian:/home/debian13# sudo sshd -V
OpenSSH_10.0p2 Debian-7+deb13u4, OpenSSL 3.5.7 9 Jun 2026
root@debian:/home/debian13# sudo sshd -T | grep -E '^(ciphers|macs|kexalgorithms|hostkeyalgorithms|pubkeyacceptedalgorithms) '
ciphers chacha20-poly1305@openssh.com,aes128-gcm@openssh.com,aes256-gcm@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr
macs umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha1-et
m@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1
kexalgorithms mlkem768x25519-sha256,sntrup761x25519-sha512,sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha
256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521
hostkeyalgorithms ssh-ed25519-cert-v01@openssh.com,ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp384-cert-v01@opens
sh.com,ecdsa-sha2-nistp521-cert-v01@openssh.com,sk-ssh-ed25519-cert-v01@openssh.com,sk-ecdsa-sha2-nistp256-cert-v01@openssh.co
m,rsa-sha2-512-cert-v01@openssh.com,rsa-sha2-256-cert-v01@openssh.com,ssh-ed25519,ecdsa-sha2-nistp256,ecdsa-sha2-nistp384,ecds
a-sha2-nistp521,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com,rsa-sha2-512,rsa-sha2-256
pubkeyacceptedalgorithms ssh-ed25519-cert-v01@openssh.com,ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp384-cert-v0
1@openssh.com,ecdsa-sha2-nistp521-cert-v01@openssh.com,sk-ssh-ed25519-cert-v01@openssh.com,sk-ecdsa-sha2-nistp256-cert-v01@ope
nssh.com,rsa-sha2-512-cert-v01@openssh.com,rsa-sha2-256-cert-v01@openssh.com,ssh-ed25519,ecdsa-sha2-nistp256,ecdsa-sha2-nistp3
84,ecdsa-sha2-nistp521,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com,rsa-sha2-512,rsa-sha2-256
root@debian:/home/debian13#
```

Wyłączymy hmac-sha1 oraz hmac-sha1-etm@openssh.com, są to starsze alg. MAC oparte na sha1 umac-64@openssh.com oraz umac64-etm@openssh.com, są to warianty z 64-bitowym tagiem który daje mniejsze bezpieczeństwo.

Dodajemy do naszego pliku konfiguracyjnego
“MACs umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512”

```
#Off old alg
MACs umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,umac-128@openssh.com,hmac-sha2-256,h
mac-sha2-512
```

Ważne dodajemy to przed “Match”

Nowa lista algorytmów MAC

```
root@debian:/home/debian13# sudo sshd -t
root@debian:/home/debian13# sudo sshd -T | grep '^macs '
macs umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,umac-128@openssh.com,hmac-sha2-256,h
mac-sha2-512
root@debian:/home/debian13#
```

Przeładuwujemy nasze zmiany

```
root@debian:/home/debian13# sudo systemctl reload ssh
root@debian:/home/debian13#
```

Ustawimy teraz niestandardowy port SSH

Aby tego dokonać przechodzimy do pliku konfiguracyjnego SSH

```
root@debian:/home/debian13# sudo nano /etc/ssh/sshd_config
```

Zmieniamy #Port 22 na dowolny wybrany przez nas port

```
  GNU nano 8.4                                        /etc/ssh/sshd_config *

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO
```

```
  GNU nano 8.4                                        /etc/ssh/sshd_config *

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2137
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO
```

Sprawdzamy poprawność konfiguracji

```
root@debian:/home/debian13# sudo sshd -t
root@debian:/home/debian13#
```

Przeładuwujemy usługę

```
root@debian:/home/debian13# sudo systemctl reload ssh
root@debian:/home/debian13#
```

Sprawdzamy czy port nasłuchuje na 2137

```
root@debian:/home/debian13# sudo ss -lntp | grep sshd
LISTEN 0      128          0.0.0.0:2137      0.0.0.0:*    users:(("sshd",pid=854,fd=6))
LISTEN 0      128             [::]:2137         [::]:*    users:(("sshd",pid=854,fd=7))
root@debian:/home/debian13#
```

Testujemy połączenie

```
+4 ❯ ssh -p 2137 user1@192.168.122.203
user1@192.168.122.203's password:
Linux debian 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Oct 27 18:02:04 2026 from 192.168.122.1
user1@debian:~$
```

Połączenie przez port 2137 działa

Natomiast na porcie 22 już nie, ponieważ zmieniliśmy go na 2137

```
+4 ❯ ssh -p 22 user1@192.168.122.203
ssh: connect to host 192.168.122.203 port 22: Connection refused

~
+4 ❯
```

Skonfigurujemy teraz blokowanie problematycznych adresów IP(niepotrafiących się poprawnie uwierzytelnić) oraz dodamy zaufany adres IP który nie będzie podlegał karom

Sprawdzamy dostępne opcje “persoruce”

```
root@debian:/home/debian13# sudo sshd -T | grep -i persource
persourcepenaltyexemptlist none
persourcemaxstartups none
persourcenetblocksize 32:128
persourcepenalties crash:90 authfail:5 noauth:1 grace-exceeded:10 refuseconnection:10 max:600 min:15 max-sources4:65536 max-so
urces6:65536 overflow:permissive overflow6:permissive
root@debian:/home/debian13#
```

Dodajemy w naszym pliku konfiguracyjnym kary za nieudane logowania oraz zaufany adres IP który nie będzie podlegał tym karom

```
#Temporary penatlies for repated failed SSH auth
PerSourcePenalties authfail:60s
PerSourceNetBlockSize 32:128

#Trusted IP excepted from penalties
PerSourcePenaltyExemptList 192.168.122.1
```

Naliczamy 60 sekund kary za nieudane logowania oraz blokujemy pojedynczy adres IP zamiast całej podsieci. Dodajemy zaufany adres IP.

Sprawdzamy konfigurację

```
root@debian:/home/debian13# sudo sshd -t
root@debian:/home/debian13# sudo sshd -T | grep -i persource
persourcepenaltyexemptlist 192.168.122.1
persourcemaxstartups none
persourcenetblocksize 32:128
persourcepenalties crash:90 authfail:60 noauth:1 grace-exceeded:10 refuseconnection:10 max:600 min:15 max-sources4:65536 max
ources6:65536 overflow:permissive overflow6:permissive
root@debian:/home/debian13#
```

Przeładuwujemy usługę

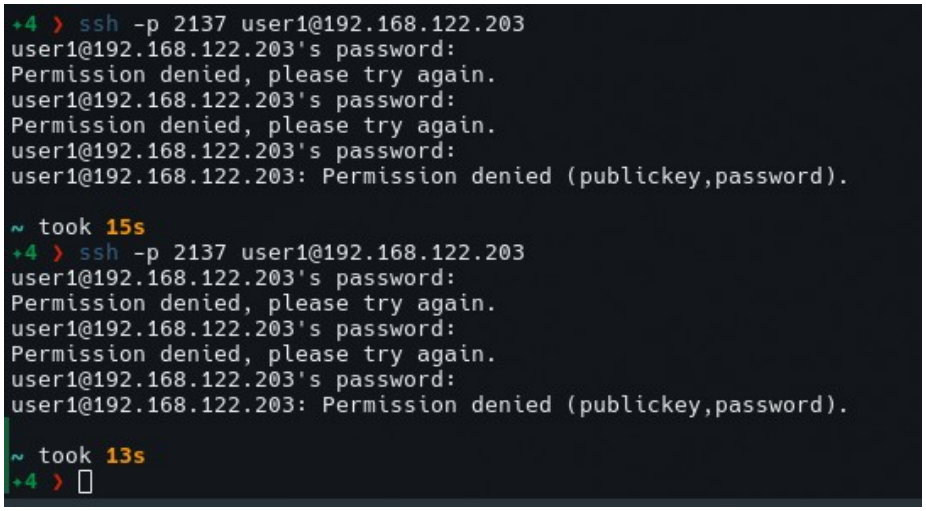
```
root@debian:/home/debian13# sudo systemctl reload ssh
root@debian:/home/debian13#
```

Testujemy czy nasze reguły działają

```
root@debian2:/home/debian2# ssh -p 2137 user2@192.168.122.203
user2@192.168.122.203's password:
Permission denied, please try again.
user2@192.168.122.203's password:
Permission denied, please try again.
user2@192.168.122.203's password:
user2@192.168.122.203: Permission denied (publickey,password).
root@debian2:/home/debian2# ssh -p 2137 user2@192.168.122.203
kex_exchange_identification: read: Connection reset by peer
Connection reset by 192.168.122.203 port 2137
root@debian2:/home/debian2#
```

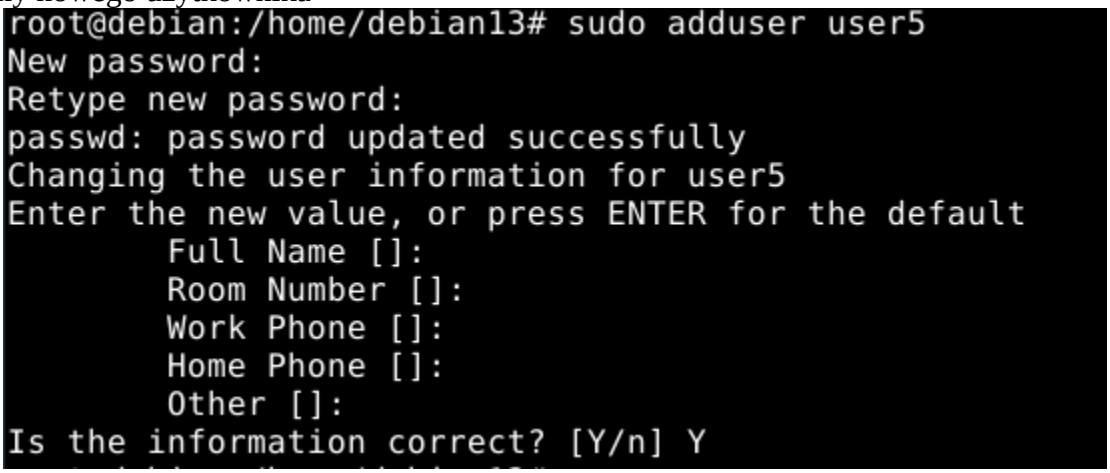
Blokada logowania

Adres który jest zaufany i nie podlega blokadzie, nie blokuję następnego połączenia po nieudanym logowaniu.



3. Ograniczenia Sudo

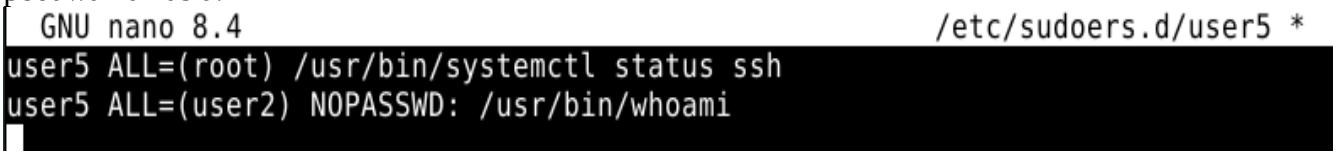
Tworzymy nowego użytkownika



Otwieramy plik sudoers.d i dodamy możliwość wykonywania komendy “systemctl status ssh” dla naszego użytkownika jako root.



Dodajemy odrazu w tym samym pliku wykonywanie komendy “whoami” jako inny użytkownik bez podawania hasła.



Przełączamy się na użytkownika “user5” oraz sprawdzamy dozwolone polecenia.

```
debian13@debian:~$ su user5
Password:
user5@debian:/home/debian13$ sudo -l
Matching Defaults entries for user5 on debian:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User user5 may run the following commands on debian:
    (root) /usr/bin/systemctl status ssh
    (user2) NOPASSWD: /usr/bin/whoami
user5@debian:/home/debian13$
```

Uruchamiamy pierwszą komendę jako root

```
user5@debian:/home/debian13$ sudo /usr/bin/systemctl status ssh
[sudo] password for user5:
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-27 15:34:06 EDT; 3h 55min ago
 Invocation: f48aafe03b1f437e9a3adc6d64f3be81
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 834 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 3997 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 3999 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 854 (sshd)
      Tasks: 1 (limit: 4634)
     Memory: 9.1M (peak: 31.4M)
        CPU: 4.355s
     CGroup: /system.slice/ssh.service
             └─854 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 27 19:11:43 debian sshd-session[4029]: User child is on pid 4063
Oct 27 19:11:43 debian sshd-session[4029]: debug3: mm_request_receive: entering
Oct 27 19:11:43 debian sshd-session[4029]: debug3: monitor_read: checking request 28
Oct 27 19:11:43 debian sshd-session[4029]: debug3: mm_answer_pty: entering
Oct 27 19:11:43 debian sshd-session[4029]: debug2: session_new: allocate (allocated 0 max 10)
Oct 27 19:11:43 debian sshd-session[4029]: debug3: session_unused: session id 0 unused
Oct 27 19:11:43 debian sshd-session[4029]: debug1: session_new: session 0
Oct 27 19:11:43 debian sshd-session[4029]: debug1: SELinux support disabled
Oct 27 19:11:43 debian sshd-session[4029]: debug3: mm_request_send: entering, type 29
Oct 27 19:11:43 debian sshd-session[4029]: debug3: mm_answer_pty: tty /dev/pts/2 ptyfd 6
user5@debian:/home/debian13$
```

Mamy poprawne działanie tej komendy.

Teraz uruchomimy druga komendę jako inny użytkownik bez podawania hasła

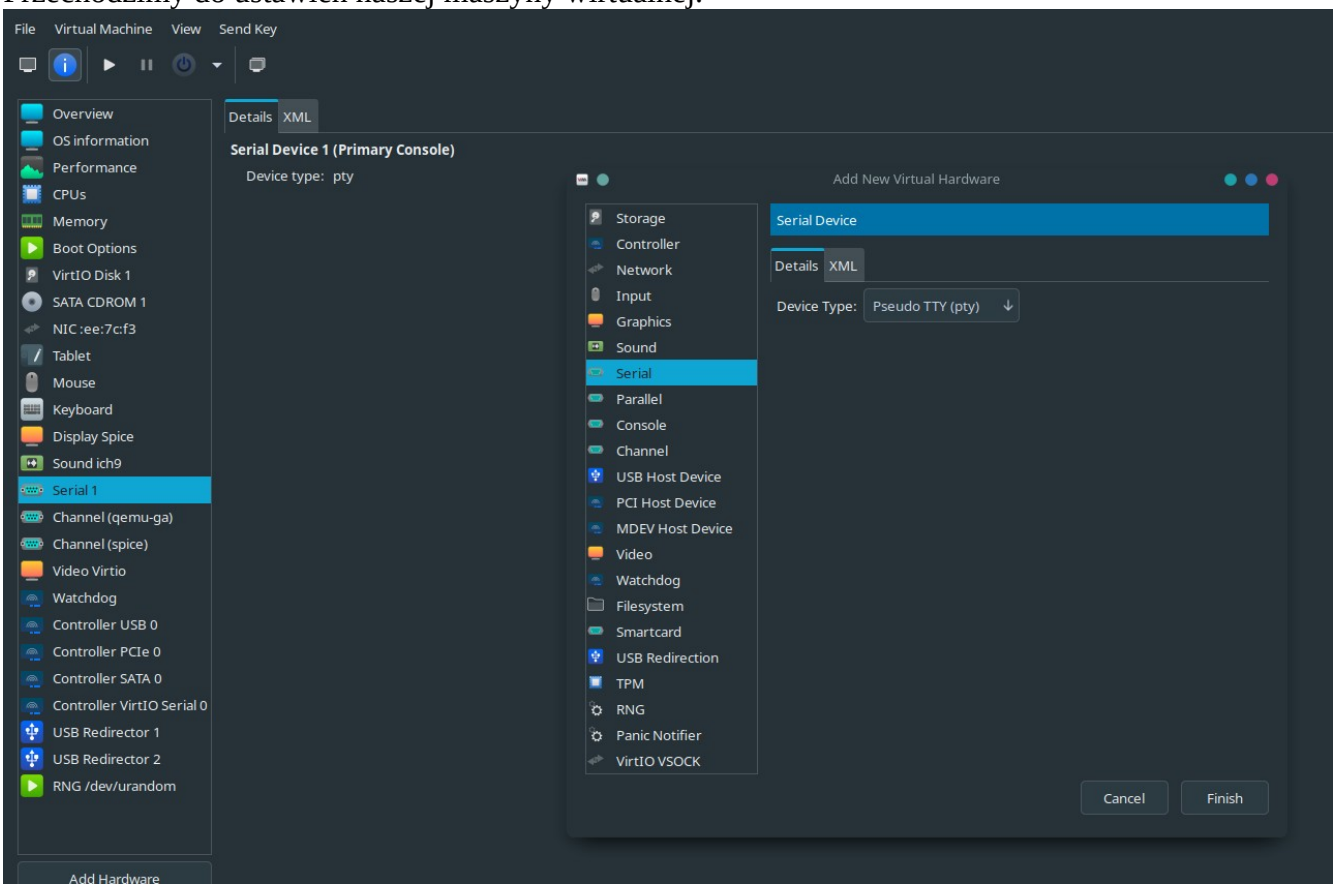
```
user5@debian:/home/debian13$ sudo -u user2 /usr/bin/whoami
user2
user5@debian:/home/debian13$
```

Działa tak jak należy, nie musieliśmy podawać hasła dla “user2” aby wykonać polecenie.

4. Alternatywne sposoby dostępu

Zaczniemy od sprawdzenia czy nasz wirtualizator wspiera dodawanie portu seryjnego aby móc skonfigurować system tak żeby wyświetlał na tym porcie konsolę.

Przechodzimy do ustawień naszej maszyny wirtualnej.



Mamy dostępny serial port, więc nie będziemy dodawać nowego, prawdopodobnie powinien działać.

Mamy już dodany port “Serial 1”, więc skorzystamy z niego.
Sprawdzamy czy usługa jest włączona.

```
root@debian:/home/debian13# systemctl is-active serial-getty@ttyS0.service
inactive
root@debian:/home/debian13#
```

Usługa jest wyłączona.

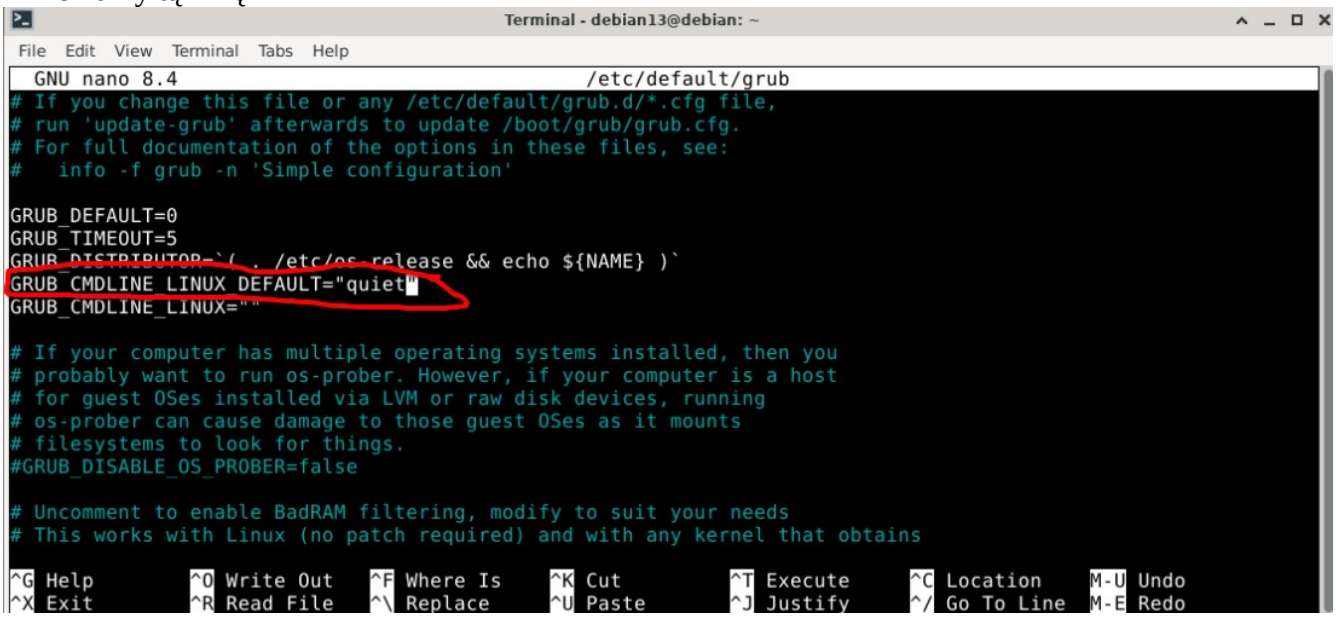
Uruchamiamy usługę konsoli i sprawdzamy jej status.

```
root@debian:/home/debian13# systemctl enable --now  serial-getty@ttyS0.service
Created symlink '/etc/systemd/system/getty.target.wants/serial-getty@ttyS0.service' → '/usr/lib/systemd/syste
m/serial-getty@.service'.
root@debian:/home/debian13# systemctl is-active serial-getty@ttyS0.service
active
root@debian:/home/debian13#
```

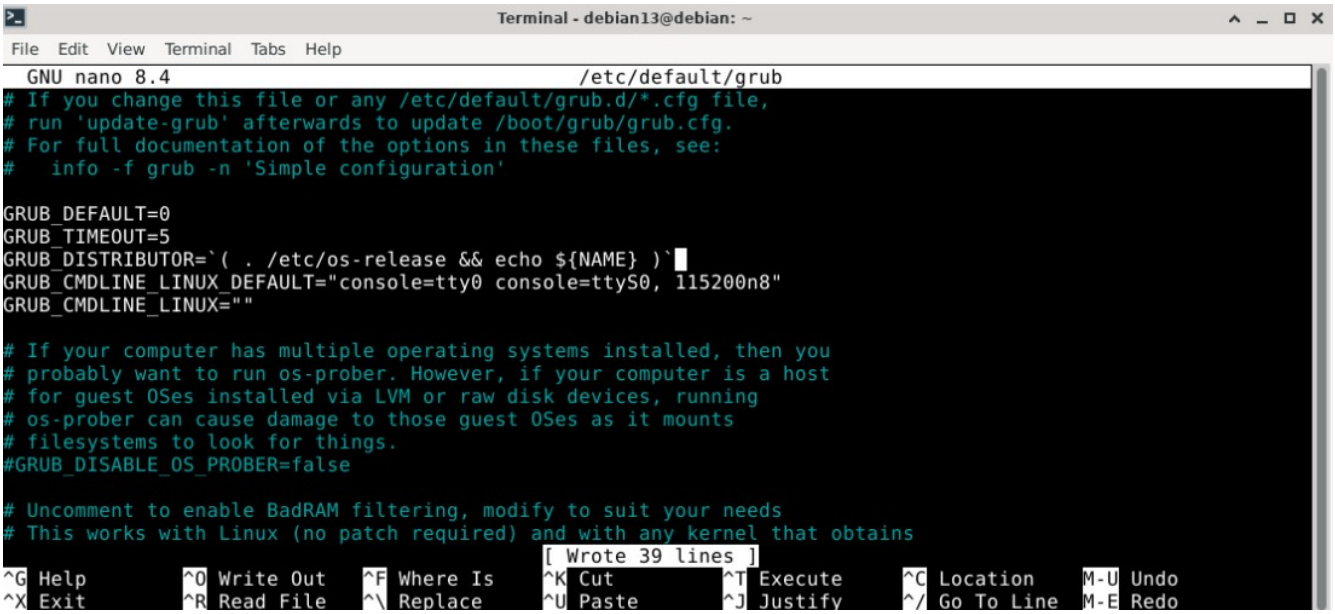
Teraz dodajemy konsole szeregową w GRUB-ie, zatem edytujemy plik grub

```
root@debian:/home/debian13# sudo nano /etc/default/grub
root@debian:/home/debian13#
```

Zmieniamy tą linię



Na tą



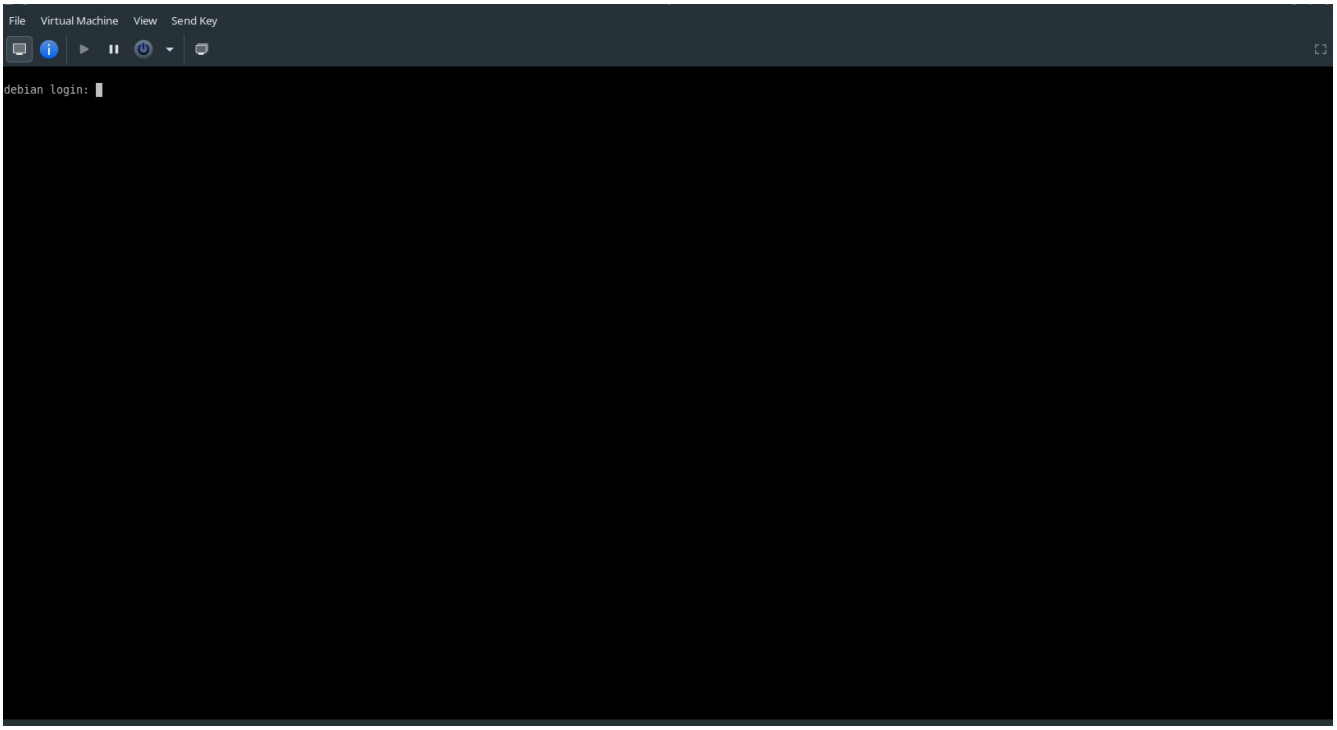
Ustawia to wyjście konsoli na ekranie wirtualnym oraz porcie szeregowym z predkością 115200
Aktualizujemy grub

```
root@debian:/home/debian13# sudo update-grub
Generating grub configuration file ...
Found background image: /usr/share/images/desktop-base/desktop-grub.png
Found linux image: /boot/vmlinuz-6.12.113+deb13-amd64
Found initrd image: /boot/initrd.img-6.12.113+deb13-amd64
Found linux image: /boot/vmlinuz-6.12.107+deb13-amd64
Found initrd image: /boot/initrd.img-6.12.107+deb13-amd64
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
root@debian:/home/debian13#
root@debian:/home/debian13#
```

Restartujemy naszą maszynę wirtualną.

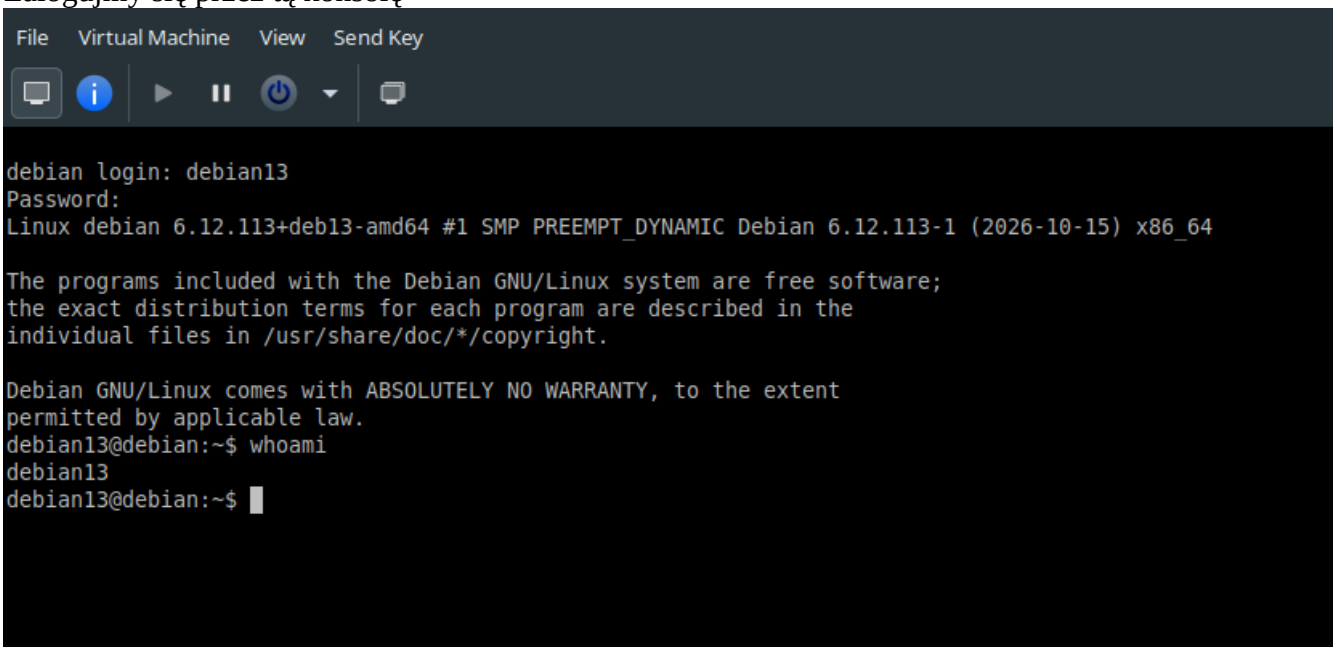
```
root@debian:/home/debian13# sudo reboot
```

Teraz w virt-manager sprawdzamy czy nasz serial port działa
Przechodzimy do zakładki view → consoles → Serial 1 i wybieramy tą opcję



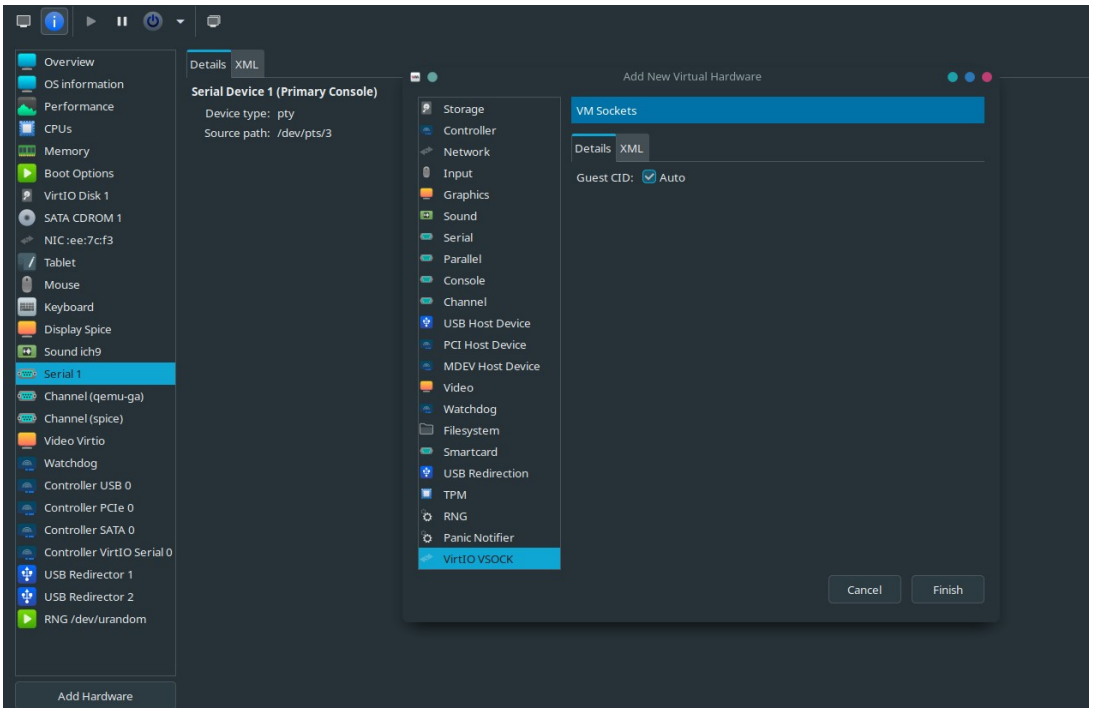
Po przełączeniu na Serial 1 mamy dostępną konsolę z logowaniem, zatem udało nam się skonfigurować wyświetlanie konsoli przez port seryjny.

Zalogujmy się przez tą konsolę



Logowanie działa wraz z konsolą.

Teraz skonfigurujemy kolejną metodę dostępu, czyli dostęp poprzez sieć VSOCK(zamiast adresów IP korzysta z Context ID, połączenie niezależne od karty sieciowej).
Dodajemy urządzenie VirtIO VSOCK.



Maszyna wirtualna musi być wyłączona!

Sprawdzamy w VM-ce czy VSOCK jest podłączony

```
debian13@debian:~$ ls -l /dev/vsock
crw-rw-rw- 1 root root 10, 261 Oct 28 08:30 /dev/vsock
debian13@debian:~$
```

```
root@debian:/home/debian13# sudo dmesg | grep -i vsock
[    5.760974] NET: Registered PF_VSOCK protocol family
[    5.864486] systemd-ssh-generator[334]: Failed to query local AF_VSOCK CID: Cannot assign requested address
root@debian:/home/debian13#
```

System jeszcze nie potrafi uzyskać lokalnego CID przez VSOCK

Restartujemy systemd-udevd(odpowiedzialne za obsługę zdarzeń dotyczące urządzeń sprzętowych) oraz ponownie uruchamiamy nasza maszynę wirtualną.

```
root@debian:/home/debian13# sudo systemctl restart systemd-udevd
root@debian:/home/debian13# sudo reboot
```

Pomimo przeprowadzonej diagnostyki nie udało mi się rozwiązać problemu z uzyskaniem prawidłowego CID przez maszynę wirtualną. W związku z tym nie udało mi się skonfigurować

połączenia SSH przez VSOCK. Zdecydowałem się nie wykonywać kolejnych poleceń bez pełnego zrozumienia ich działania.

Skonfigurujemy teraz administracyjny panel webowy “cockpit”

Instalujemy panel

```
root@debian:/home/debian13# sudo apt update
Get:1 http://security.debian.org/debian-security trixie-security InRelease [43.4 kB]
Hit:2 http://deb.debian.org/debian trixie InRelease
Get:3 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:4 http://security.debian.org/debian-security trixie-security/main Sources [239 kB]
Get:5 http://security.debian.org/debian-security trixie-security/main amd64 Packages [270 kB
Fetched 600 kB in 0s (1,647 kB/s)
All packages are up to date.
root@debian:/home/debian13# sudo apt install cockpit
Installing:
  cockpit

Installing dependencies:
  appstream               cockpit-packagekit  cockpit-ws        libcrack2            libpwqu
  cockpit-bridge          cockpit-storaged    cracklib-runtime  libpwquality-common  libstem
  cockpit-networkmanager  cockpit-system      libappstream5     libpwquality-tools   libxmlb

Suggested packages:
  apt-config-icons  cockpit-sosreport  udisks2-btrfs  mdadm     sssd-dbus
  cockpit-doc       python3-pcp        udisks2-lvm2   lastlog2

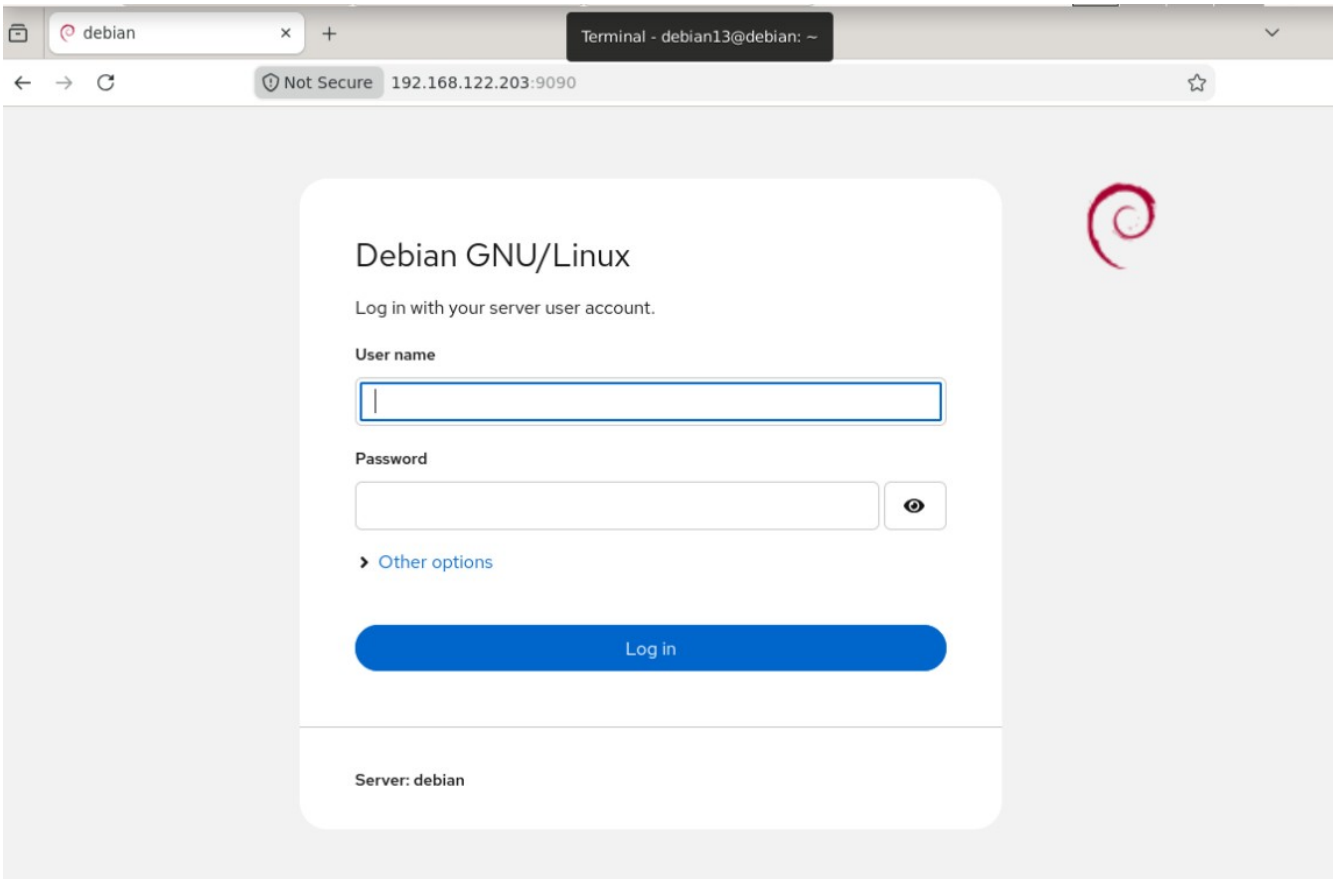
Summary:
  Upgrading: 0, Installing: 18, Removing: 0, Not Upgrading: 0
```

Uruchamiamy usługę i sprawdzamy czy jest aktywna

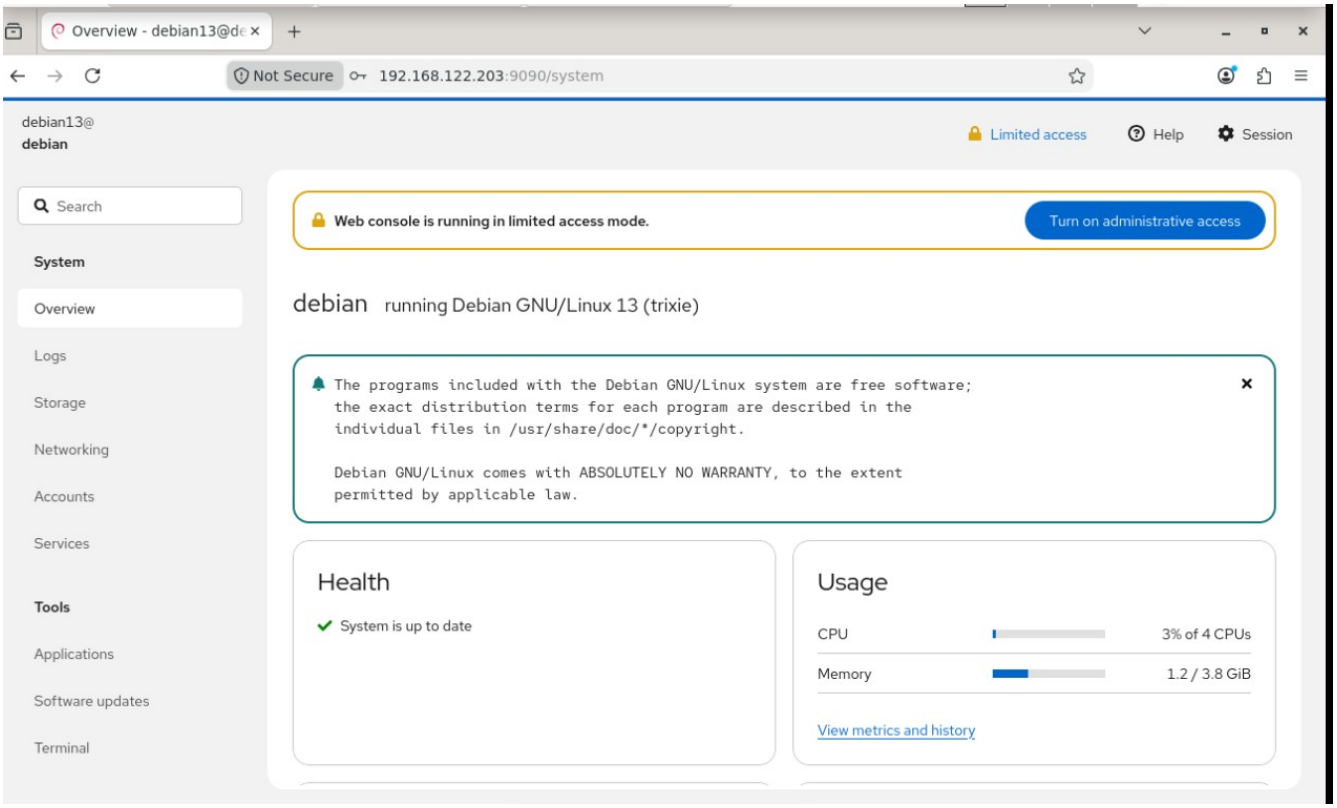
```
root@debian:/home/debian13# sudo systemctl enable --now cockpit.socket
root@debian:/home/debian13# systemctl status cockpit-socket --no-pager
Unit cockpit-socket.service could not be found.
root@debian:/home/debian13# systemctl status cockpit.socket --no-pager
● cockpit.socket - Cockpit Web Service Socket
     Loaded: loaded (/usr/lib/systemd/system/cockpit.socket; enabled; preset: enabled)
     Active: active (listening) since Wed 2026-10-28 08:58:37 EDT; 2min 47s ago
 Invocation: 67a7af89a3ec47c987494750faa6401d
   Triggers: ● cockpit.service
       Docs: man:cockpit-ws(8)
     Listen: [::]:9090 (Stream)
      Tasks: 0 (limit: 4634)
     Memory: 88K (peak: 2.6M)
        CPU: 24ms
     CGroup: /system.slice/cockpit.socket

Oct 28 08:58:37 debian systemd[1]: Starting cockpit.socket - Cockpit Web Service Socket...
Oct 28 08:58:37 debian systemd[1]: Listening on cockpit.socket - Cockpit Web Service Socket.
root@debian:/home/debian13#
```

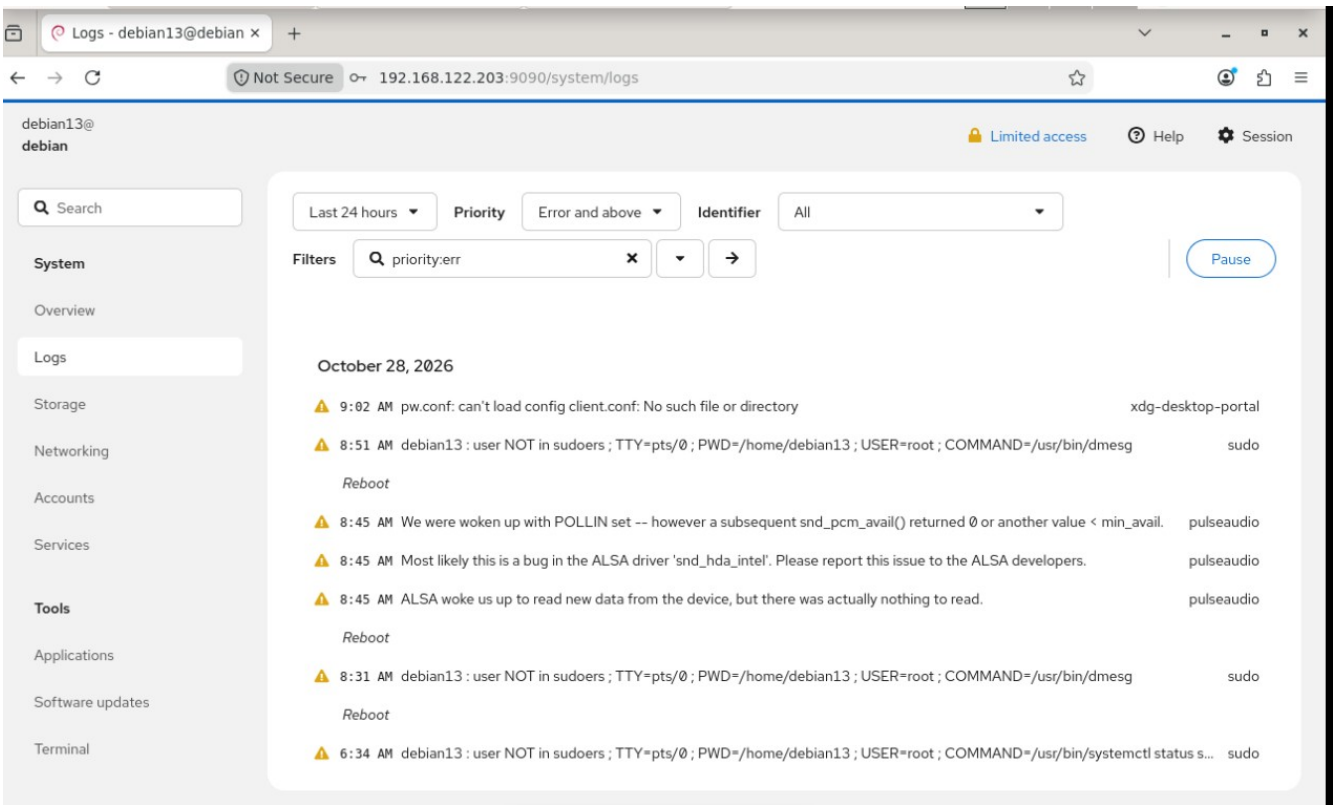
Cockpit domyślnie działa na porcie 9090 poprzez HTTPS, zatem otwieramy w przeglądarce nasz panel



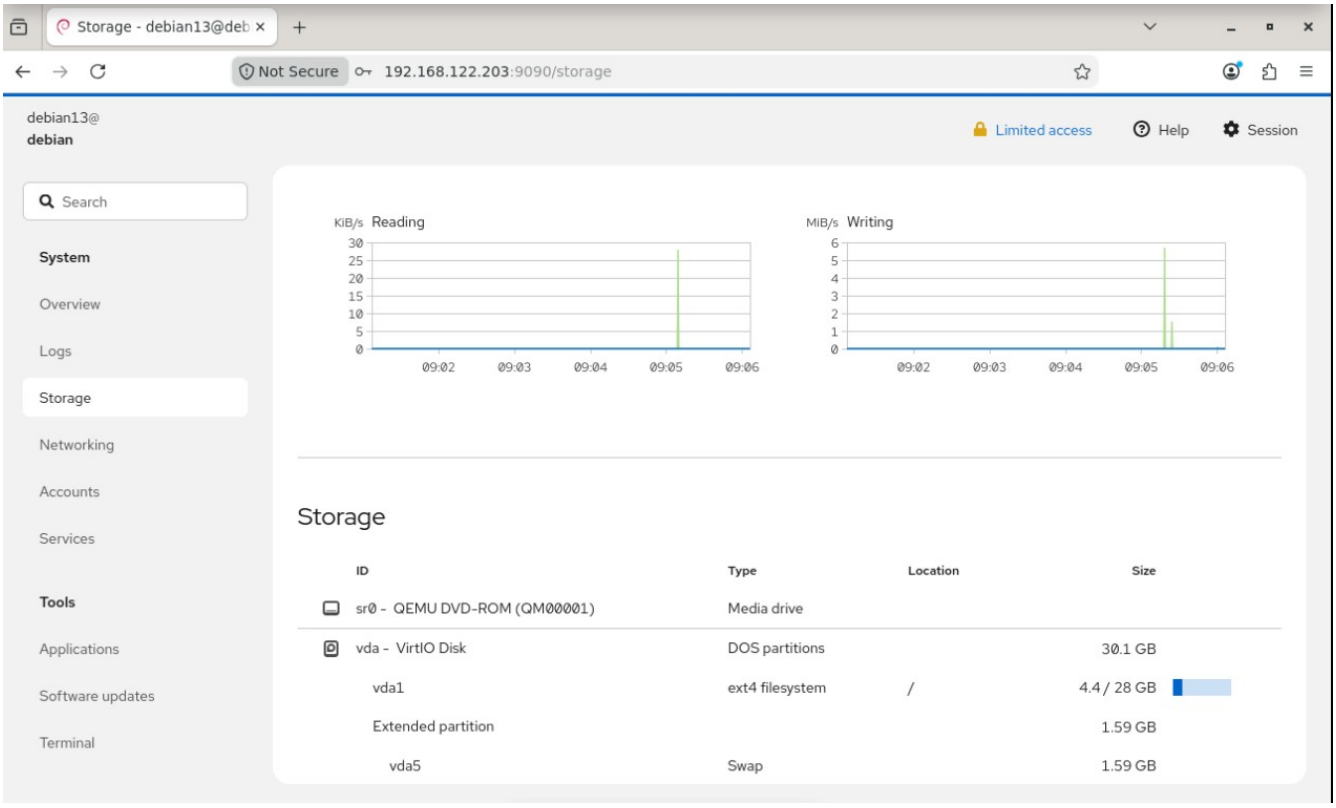
Zalogujemy się oraz sprawdzimy kilka funkcji



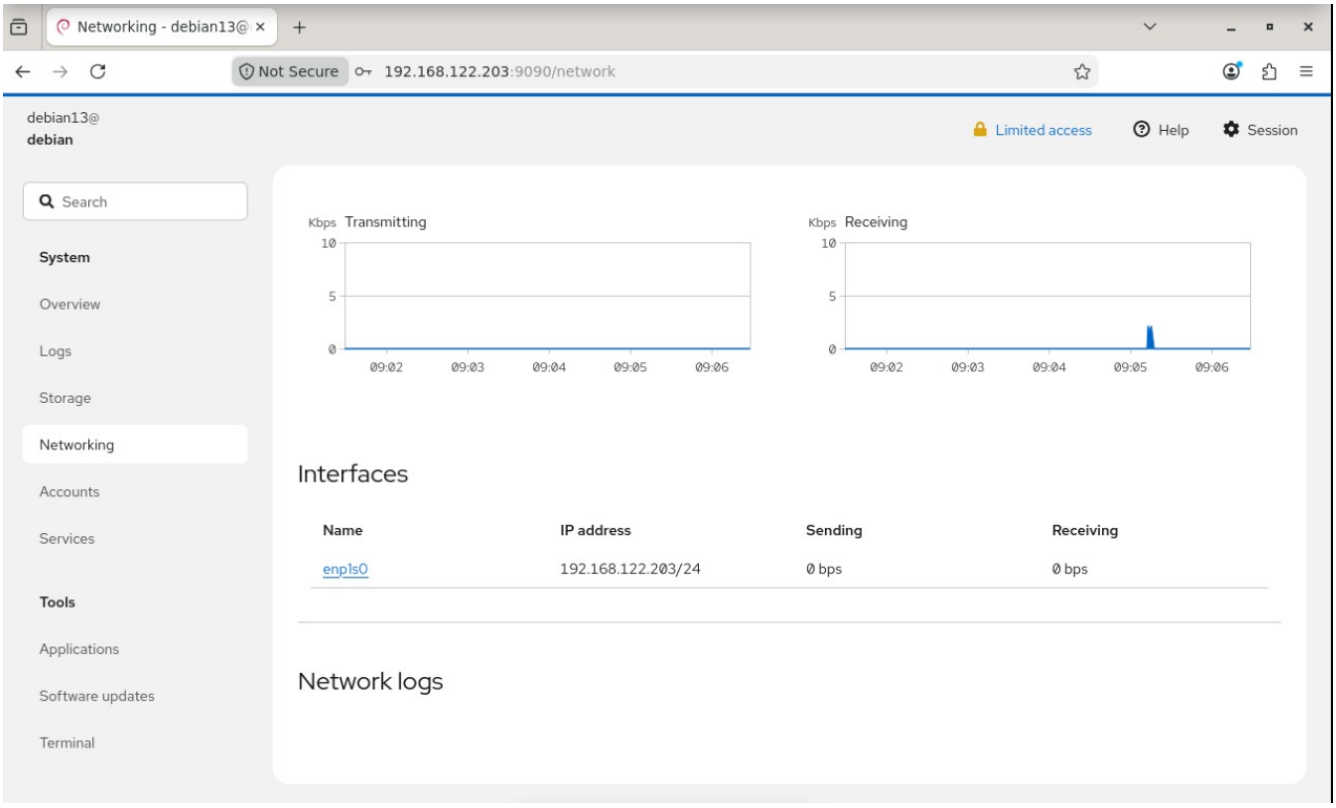
Zakładka Overview – stan systemu, wykorzystanie CPU i pamięci



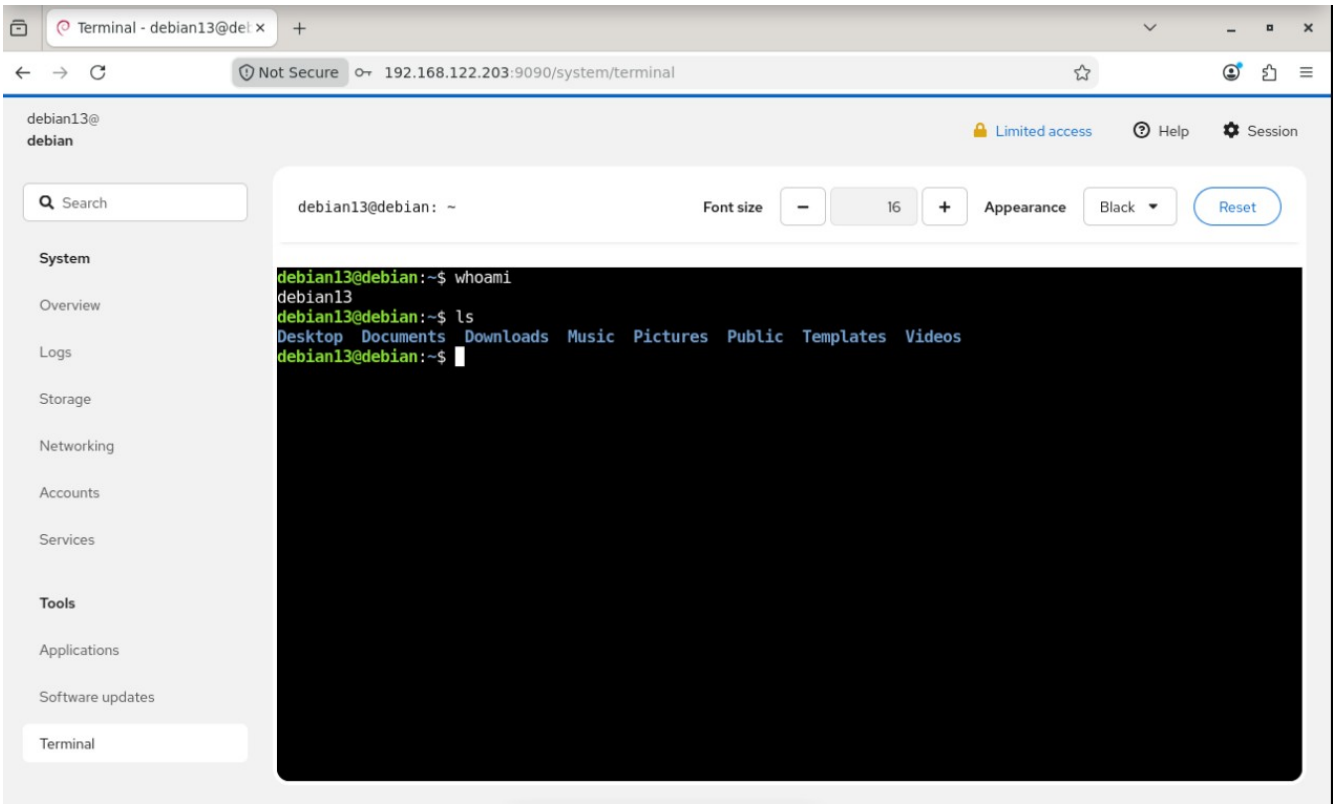
Zakładka Logs – logi systemowe



Zakładka Storage – dyski w systemie i ich wykorzystanie



Zakładka Networking – interfejsy i konfiguracja sieci



Zakładka tools->Terminal - mamy do dyspozycji terminal administracyjny z poziomu przeglądarki

5. Podsumowanie

Zadanie nauczyło mnie konfiguracji zaawansowanej konfiguracji ssh oraz alternatywnych sposobów dostępu do maszyny wirtualnej.

Stworzyliśmy nowego użytkownika z uprawnieniami sudo i skonfigurowaliśmy logowanie asymetrczyne, pokazaliśmy działanie tego logowania.

Zrobiliśmy zaawansowaną konfigurację ssh, poprzez stworzenie grup dla danych użytkowników aby przypisać każdemu różne funkcjonalności, wyłączyliśmy przestarzałe algorytmy kryptograficzne oraz ustawiliśmy niestandardowy port ssh. Dodaliśmy również blokowanie problematycznych adresów IP, jak i zaufane adresy IP które nie podlegały blokowaniu.

Utworzyliśmy nowego użytkownika który miał prawo wykonywać tylko jedną komendę z uprawnieniami sudo i jedną komendę jako inny użytkownik bez podawania hasła, po czym zaprezentowaliśmy działanie dokonanych zmian.

Skonfigurowaliśmy alternatywne sposoby dostępu do maszyny wirtualnej, zaczęliśmy od portu seryjnego na którym wyświetliliśmy konsolę naszej maszyny wirtualnej.
Następnie spróbowaliśmy dołączyć maszynę do sieci VSOCK, niestety nie udało się tego dokonać.
Na koniec skonfigurowaliśmy webowy panel administracyjny “Cockpit” i zaprezentowaliśmy jego działanie oraz możliwości.

Środowisko wykorzystane do realizacji zadania
-Debian 13.7.0
-Virt-manager 5.0.0
-Cockpit 337

Źródła:
https://man7.org/linux/man-pages/man1/ssh.1.html
https://oneuptime.com/blog/post/2026-03-02-restrict-ssh-access-specific-users-groups-ubuntu/view
https://www.digitalocean.com/community/tutorials/how-to-create-a-new-sudo-enabled-user-on-ubuntu#step-2-adding-a-new-user-to-the-system
https://docs.cockpit-project.org/cockpit-guide/latest/guide/startup.html

Autor: Bartosz Marcińczyk